```
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(1)
])
```

Using Early Stopping

Early stopping halts training when the validation loss stops improving, preventing overfitting.

```
from tensorflow.keras.callbacks import EarlyStopping early_stopping =
EarlyStopping(monitor='val_loss', patience=5) history = model.fit(
    X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, callbacks=[early_stopping]
)
```

Visualizing Training Results

Plot the training and validation loss to observe the model's learning process.

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'], label='Training Loss') plt.plot(history.history['val_loss'],
label='Validation Loss') plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

By following this guide, you can build and train a simple neural network using TensorFlow and Keras. This example serves as a foundation for exploring more complex architectures and tasks. Experiment with different configurations, optimizers, and hyperparameters to improve model performance.

Convolutional Neural Networks (CNNs) for Image Classification

Convolutional Neural Networks (CNNs) are a class of deep learning models specifically designed to process and analyze visual data. They are widely used in computer vision tasks such as image classification, object detection, and image segmentation. CNNs utilize specialized layers to extract and learn spatial hierarchies of features from input images, making them particularly effective for handling high-dimensional data like images.

Key Components of CNNs

1. **Convolutional Layers**: